

# Week 11 Lecture

Welcome! Today you will learn:

- Fractals and Self-Similarity
  - Functions and combinations of functions as an example of self-similarity
  - Perlin noise: Concept
  - Perlin noise in 1D and 2D
  - The use of Perlin noise in design and art
- 

## 1. Fractals and Self-Similarity

Fractals in mathematics are self-similar structures - they repeat their structures over many scales. They are useful in describing the structural organisation and shapes of many natural objects. Some examples are below:

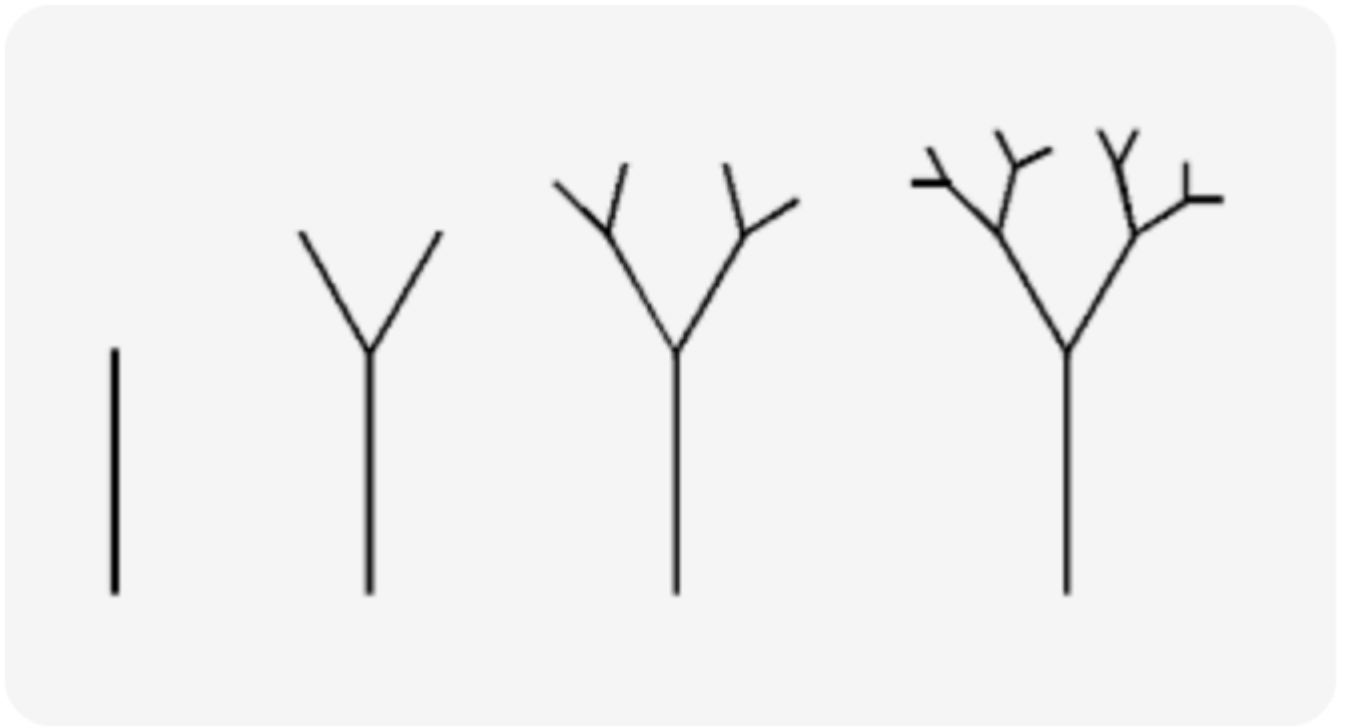


The regular ordinary cauliflower and broccoli have a fractal structure - can you see the patterns repeating at different scales?

Some other examples are ferns, leaf veins, and trees.



In all of these examples, self-similarity means that the larger object contains miniature versions of its own self at smaller and smaller scales. Consider the idea mathematically. We are all familiar with the regular series we learnt in school: Powers of 2. This goes as 1, 2, 4, 8, 16, and so on. Look at the visual now: can you see how each time a branch ends in two - we are reproducing the above series? Cutting the "tree" along any branch will produce another exact copy of the series!

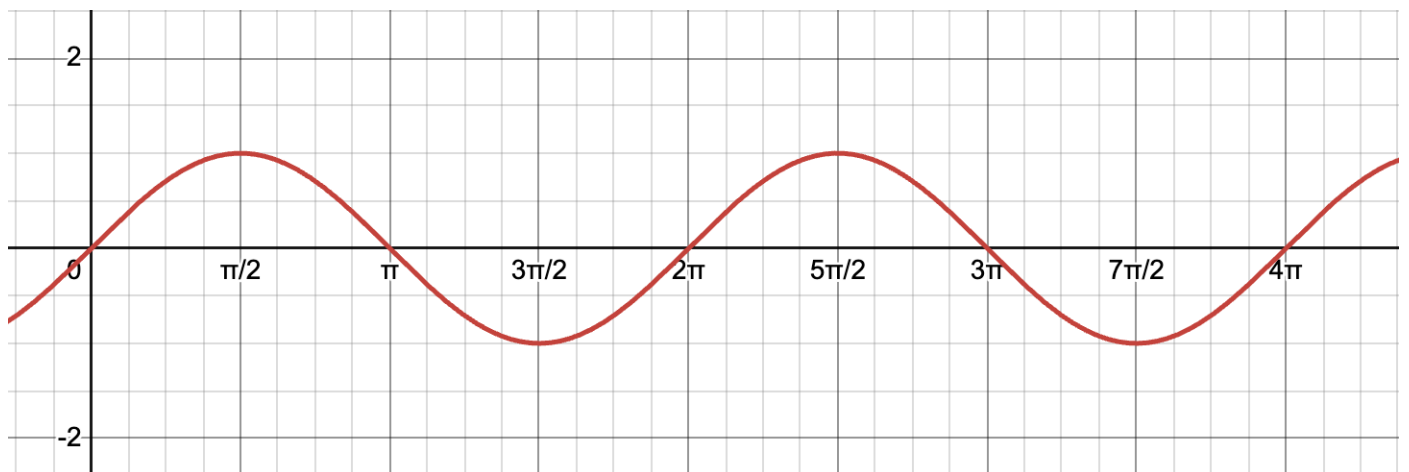


## 2. Functions

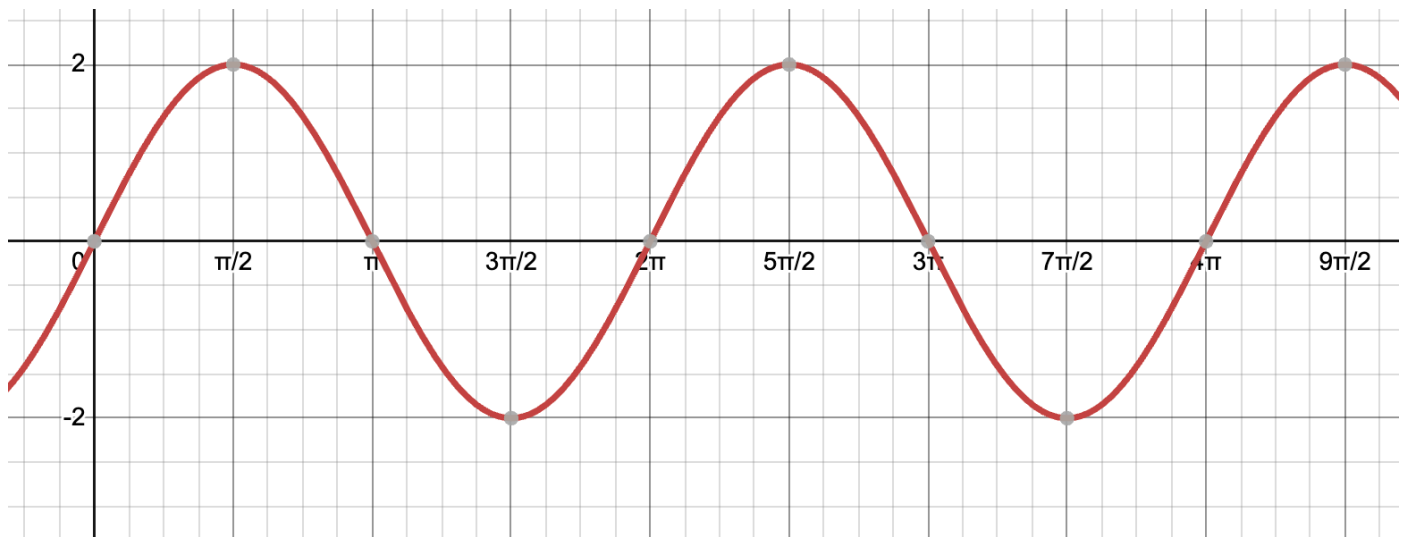
Let's look at how the above idea can now be used to combine mathematical functions: a kind of idea where a "carrier" function can contain copies of other functions. Let's start with our ordinary sin function. A "normal" sin function is described by:

$$y = \sin x$$

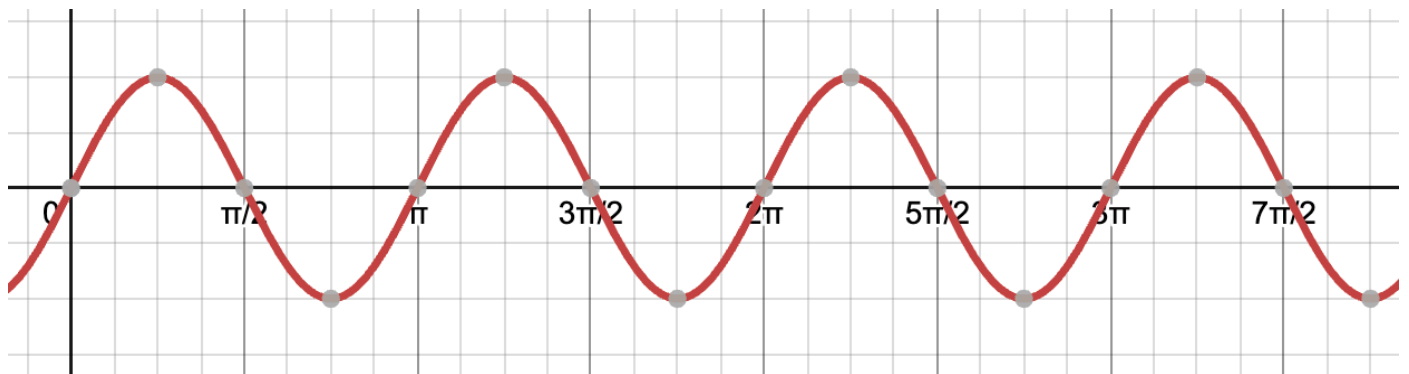
Here,  $x$  is an angle, and  $y$  is the sine of that angle. If we plot this, this is how it looks:



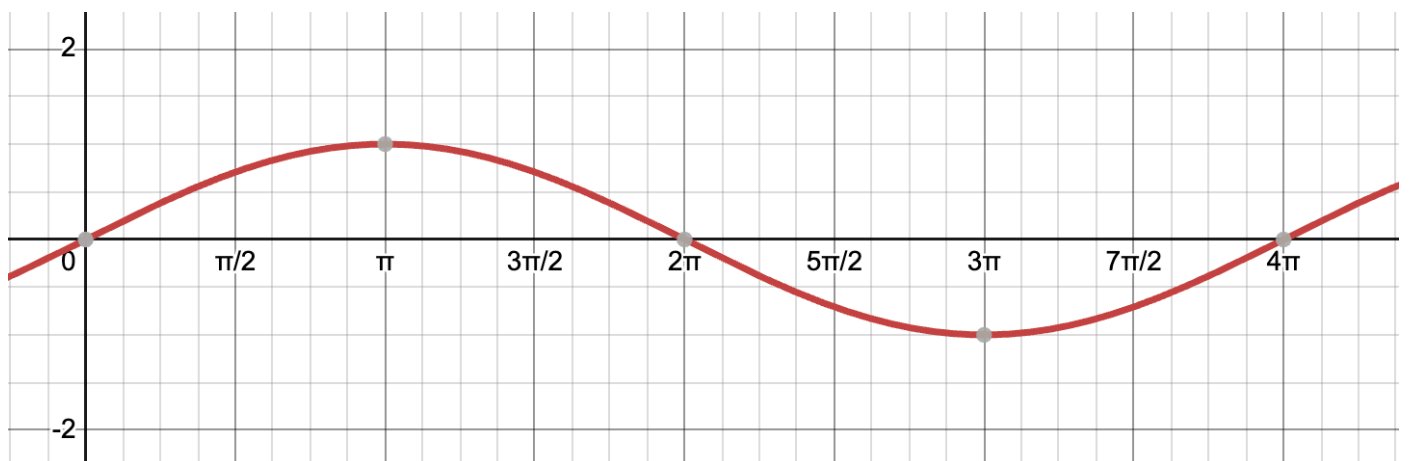
If the equation is  $y = 2 \sin x$ , the AMPLITUDE increases:



If the equation is  $y = \sin 2x$ , the FREQUENCY doubles - we get two complete waves in the space of 1.

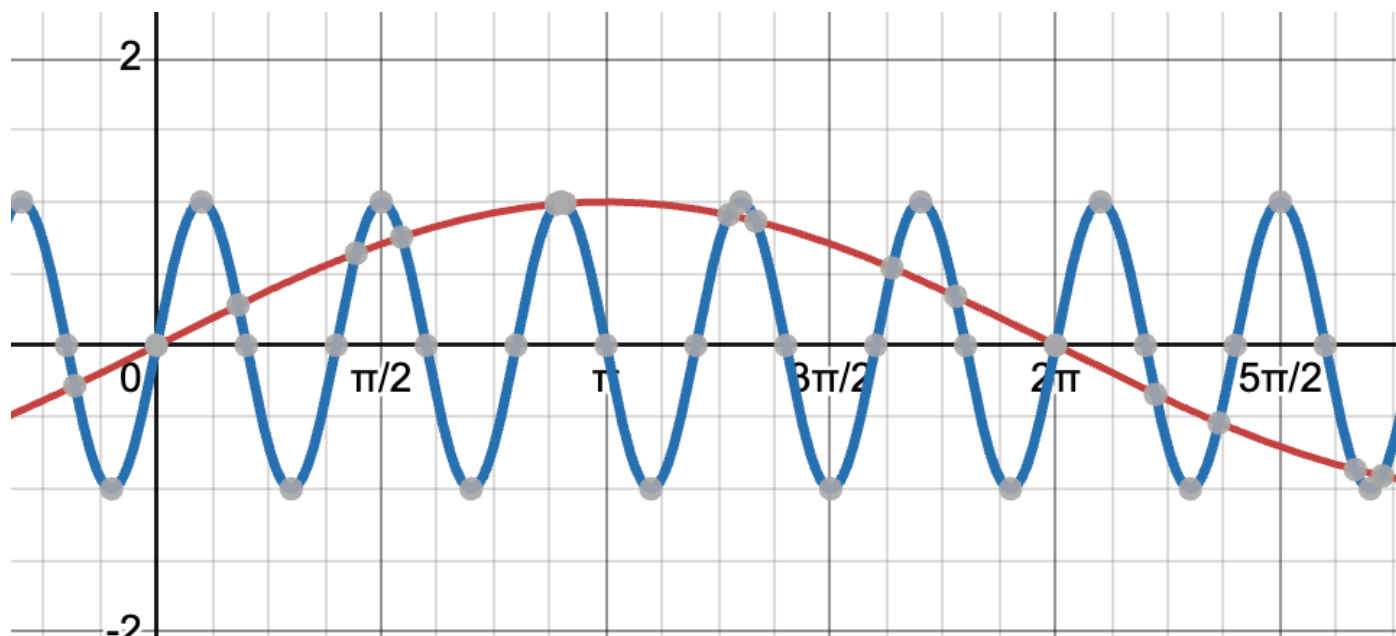


If the equation is  $y = \sin 0.5x$ , the frequency reduces to half - we get 1/2 a wave in the space of 1 wave:

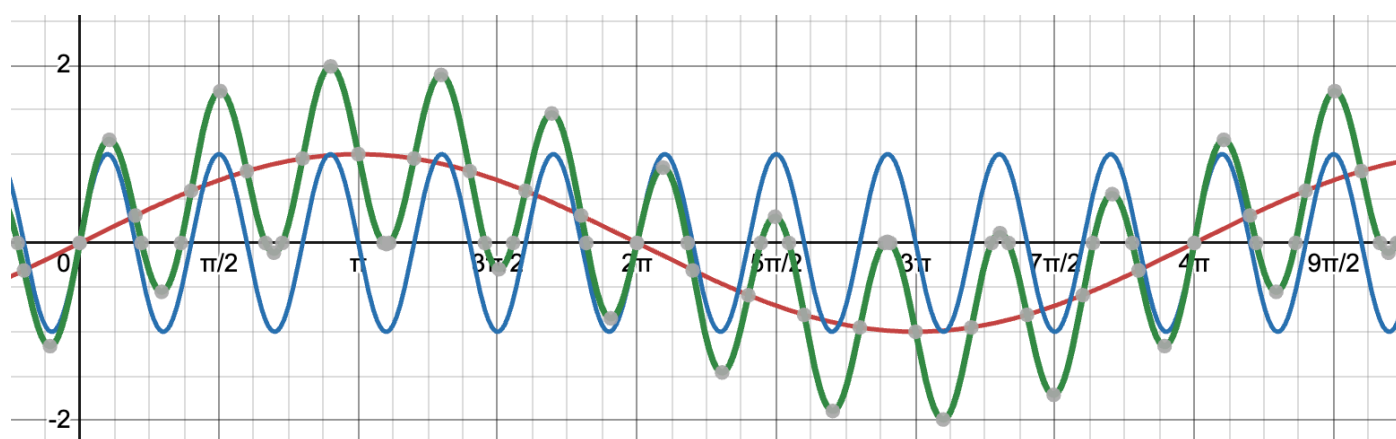


Let's now plot two waves together, a very low frequency one, and a high frequency one:

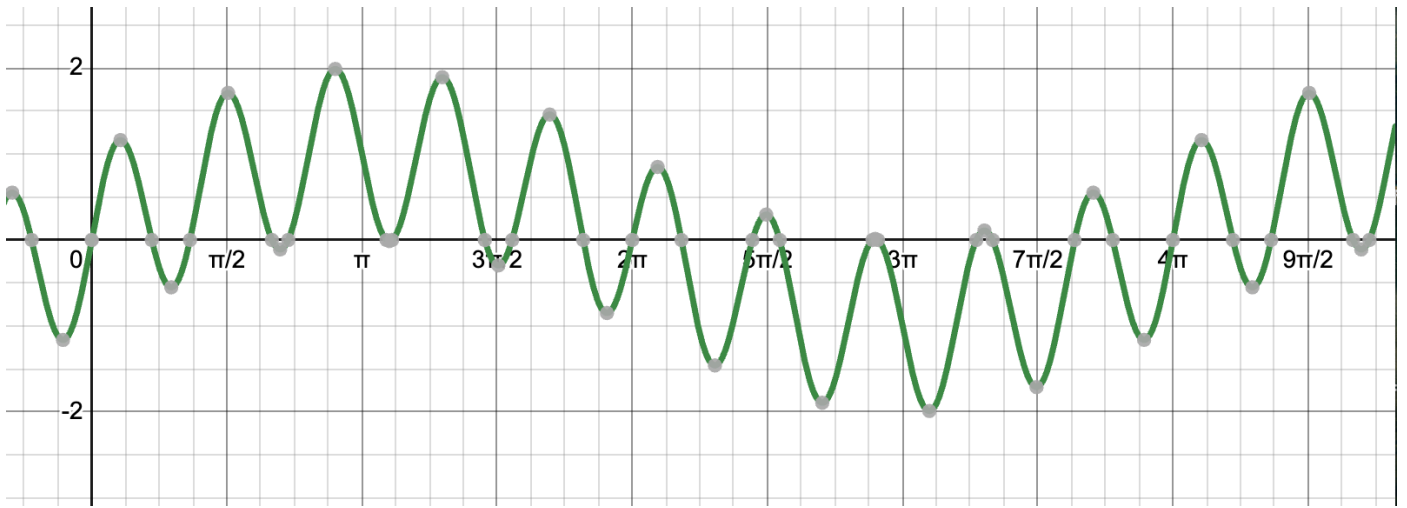




What happens now if I add these two functions? That is, now each  $y$  value is the sum of the values of the two functions at every  $x$ :



If I remove the two generator waves and keep only the result, you can see its kind of the larger wave is "carrying" copies of the smaller wave inside itself. This is the basic idea of perturbing functions with other functions that we will see in our next topic, Perlin noise. But, here, just first understand the basic concept, that given an original function, what I have done is used another function to systematically change the values of the first function by just a little amount. This gives me something "similar" to the first function, but it has some information from the second function too that is "perturbing" the first function's shape, so that the first function has smoothly morphed into a new function.



### 3. Perlin noise: Concepts

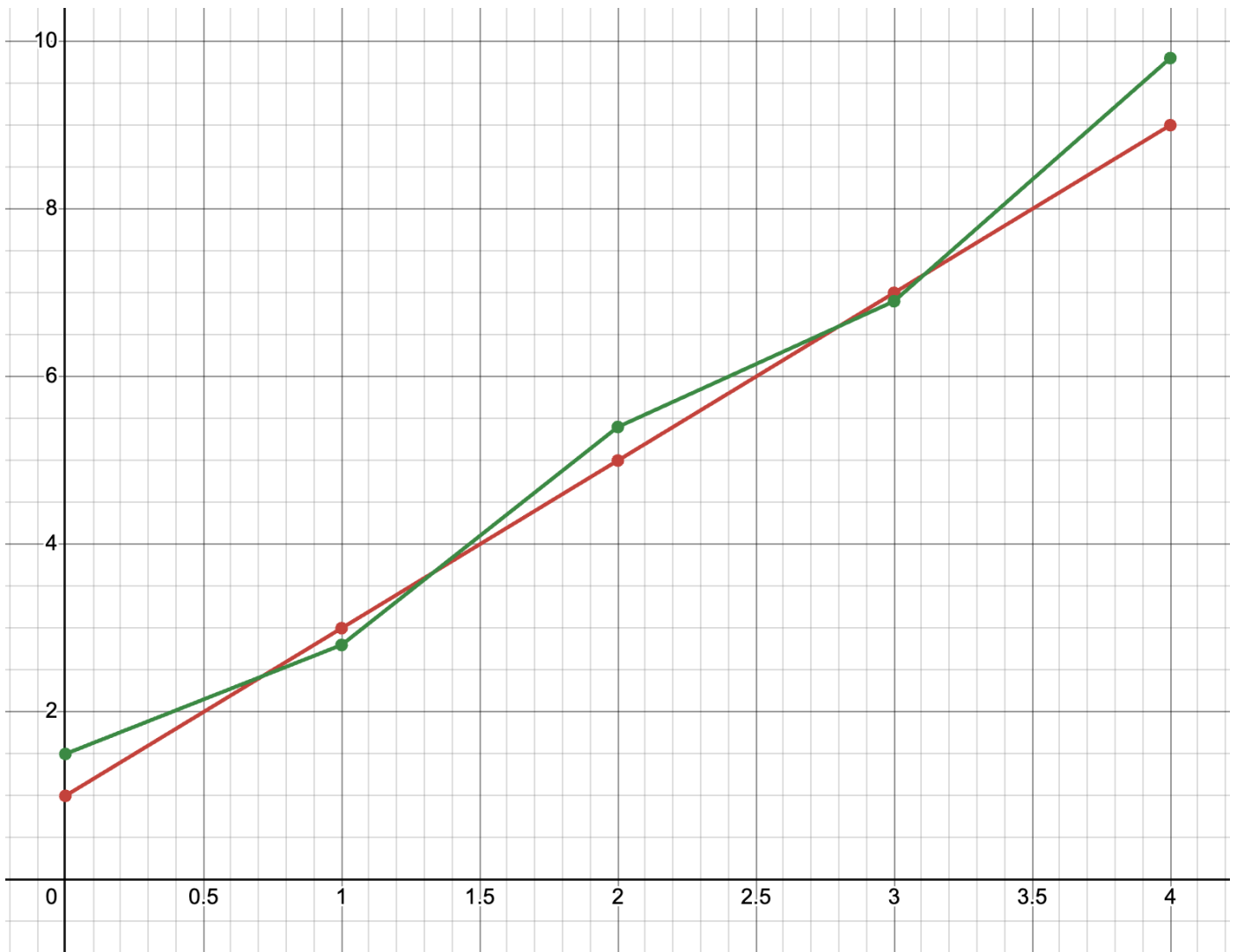
Perlin noise is a way of generating "smooth" randomness. It was introduced by Ken Perlin in the 1980s to produce realistic looking textures for Computer Generated Graphics in movies. He won an Academy Award for developing and using Perlin noise in Computer Graphics in the movie Tron. You can read about a short introduction here: [https://en.wikipedia.org/wiki/Perlin\\_noise](https://en.wikipedia.org/wiki/Perlin_noise)

↳ ([https://en.wikipedia.org/wiki/Perlin\\_noise](https://en.wikipedia.org/wiki/Perlin_noise)) and here is Ken Perlin's original SIGGRAPH paper:

<https://dl.acm.org/doi/pdf/10.1145/325165.325247> ↳

(<https://dl.acm.org/doi/pdf/10.1145/325165.325247>)

Let's go back to the basic idea of what we were discussing in the functions section above. Look at this vector:  $x = [1, 3, 5, 7, 9]$ . This is our base data. Now, if I introduce a little bit of random noise to it, I may get something like:  $x = [1.5, 2.8, 5.4, 6.9, 9.8]$ . If I draw these out, here is what they look like: can you see how I have added a little bit of noise to each point?

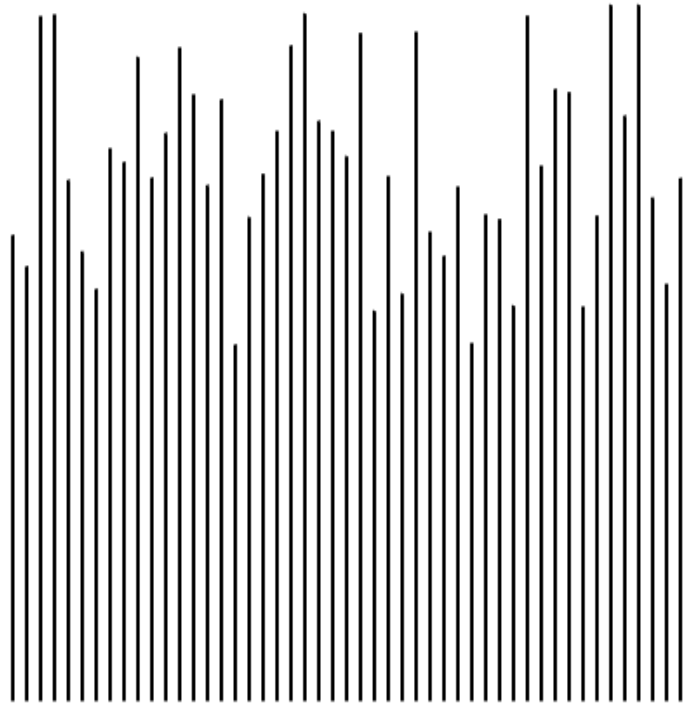


The main thing to note here, that these perturbations to the original values are random - they are **UNCORRELATED** to each other. The  $1+0.5=1.5$  and  $3-0.2=2.8$  are not related to each other, and are generated independent of each other. This is what the `random()` function does in p5.

You have already seen the `random()` function. If I generate 50 random numbers and plot them as lines in p5, here is what I get:

```
let randomNumberArray = [];  
  
function setup() {  
  createCanvas(400, 400);  
  
  for (let i = 0; i < 50; i++) {  
    randomNumberArray.push(random(0, 100));  
  }  
  
  print(randomNumberArray);  
}  
  
function draw() {  
  background(255);  
  
  for (let i = 0; i < 50; i++) {  
    line(i*4+100, 300, i*4+100, randomNumberArray[i]+100);  
  }  
}
```

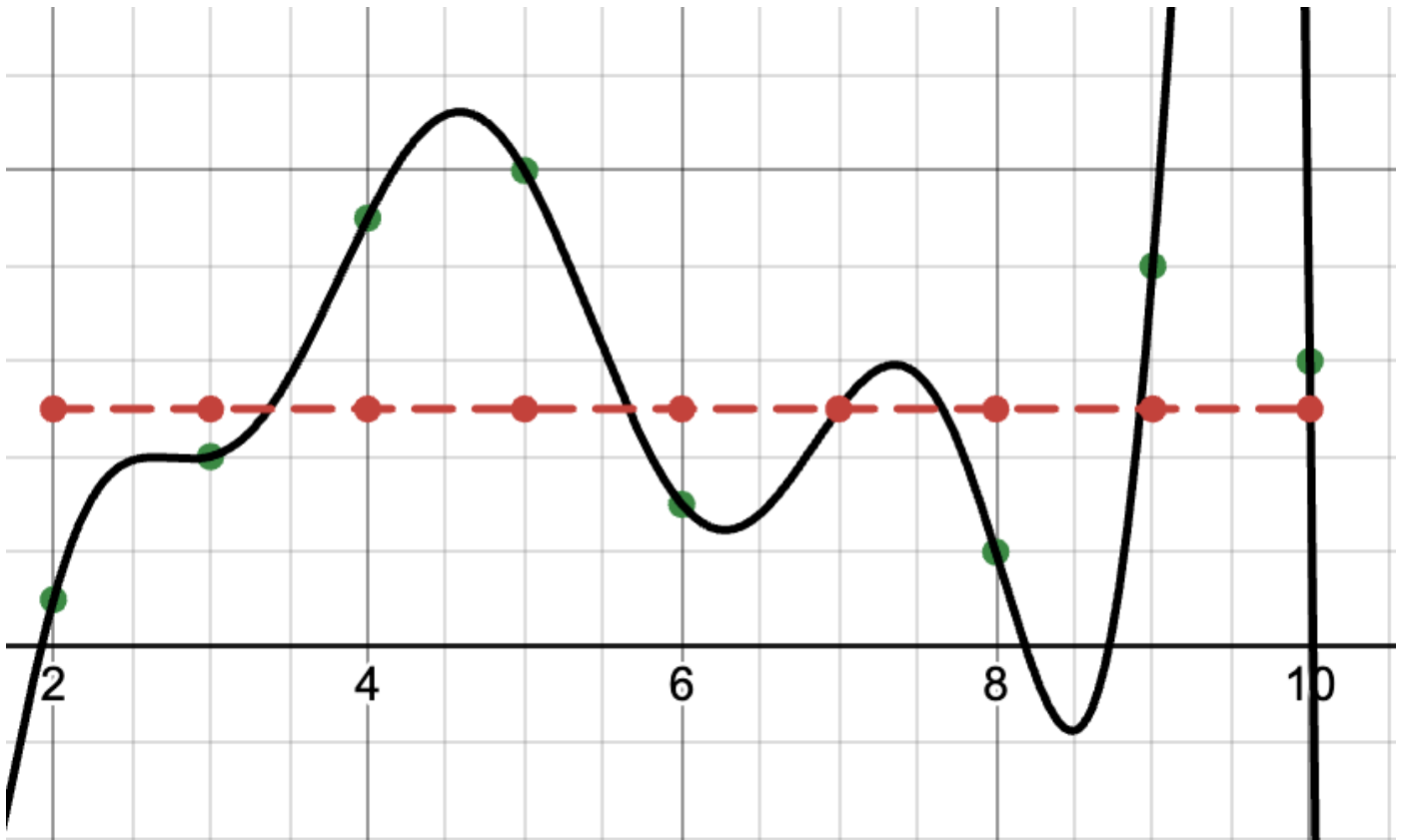




Do you see how "rough" the randomness is? Because random numbers can come from anywhere between 0 and 100 in the above example, the underlying distribution is UNIFORM: each number in the range has an equal probability of being chosen.

But what if we wanted a smoother transition. Consider again the data

$x = [5, 5, 5, 5, 5, 5, 5, 5, 5]$ . But, now, instead of adding random uncorrelated perturbations, or offsets, I add correlated ones, which means the second value is kind of related to the neighbouring first and third values. So, maybe something like:  $x = [1, 4, 9, 10, 3, 5, 2, 8, 6]$ . I have basically systematically introduced the randomness. When I plot it, and use a non-linear interpolation regression, it looks like this:



Do you see what I have done? I have created a "relationship" between neighbouring values, so that they are not uncorrelated anymore, but are correlated.

In order to introduce this type of smoothness, we use the Perlin Noise function, simply called `noise()`. Each time we generate a sketch, the `noise(x)` function returns the same value, but each different `x` returns a different value. For example, in one run, `noise(100)` returns `0.5374307048911844` and `noise(101)` returns `0.5478897676110093`, etc. Let's see what this does in code.

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
  createCanvas(400, 400);

  for (let i = 0; i < 50; i++) {
    randomNumberArray.push(random(0, 100));
  }

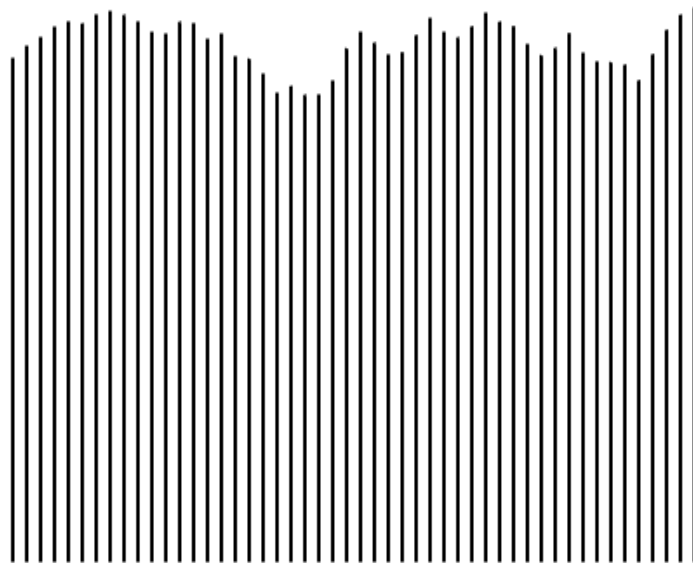
  for (let i = 0; i < 50; i += perlinNoiseStep) {
    perlinNoiseArray.push(noise(i));
  }

  print(perlinNoiseArray);
}

function draw() {
  background(255);

  for (let i = 0; i < 50; i++) {
    line(i*4+100, 300, i*4+100, perlinNoiseArray[i]*100+100);
  }
}
```

And now we get:



Do you see how the randomness is now smoothed? Perlin noise is generated with correlated values - such that even though the values are random, nearby values are still close to each other. This is an example of spatially auto-correlated randomness, when values near to each other only vary by small amounts.

## 4. Perlin noise in 1D and 2D

So far, what we have looked at is to generate a 1-dimensional perlin noise. Let's look at an application example in 1-dimension, from the Coding Train:

<https://www.youtube.com/watch?v=y7sgcFhk6ZM&list=PLRqwX-V7Uu6bgPNQAdxQZpJuJCjeOr7VD&index=3> → <https://www.youtube.com/watch?v=y7sgcFhk6ZM&list=PLRqwX-V7Uu6bgPNQAdxQZpJuJCjeOr7VD&index=3>



(<https://www.youtube.com/watch?v=y7sgcFhk6ZM&list=PLRqwX-V7Uu6bgPNQAdxQZpJuJCjeOr7VD&index=3>)

> sketch.js •

```
1 function setup() {
2   createCanvas(400, 400);
3 }
4
5 function draw() {
6   background(25);
7
8   for (let x = 0; x < width; x++) {
9     stroke(255);
10    strokeWeight(2);
11    point(x, random(height));
12  }
13
14  noLoop();
15 }
```

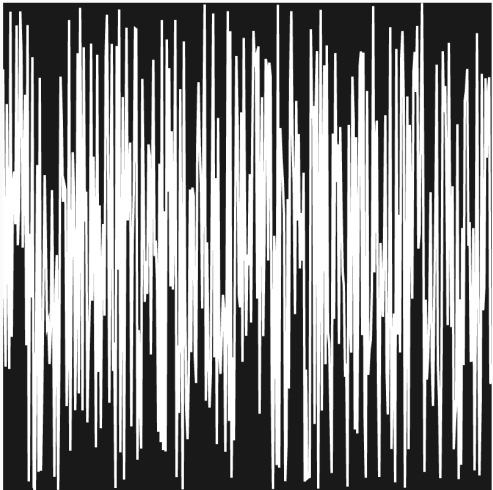
Console Clear ▼

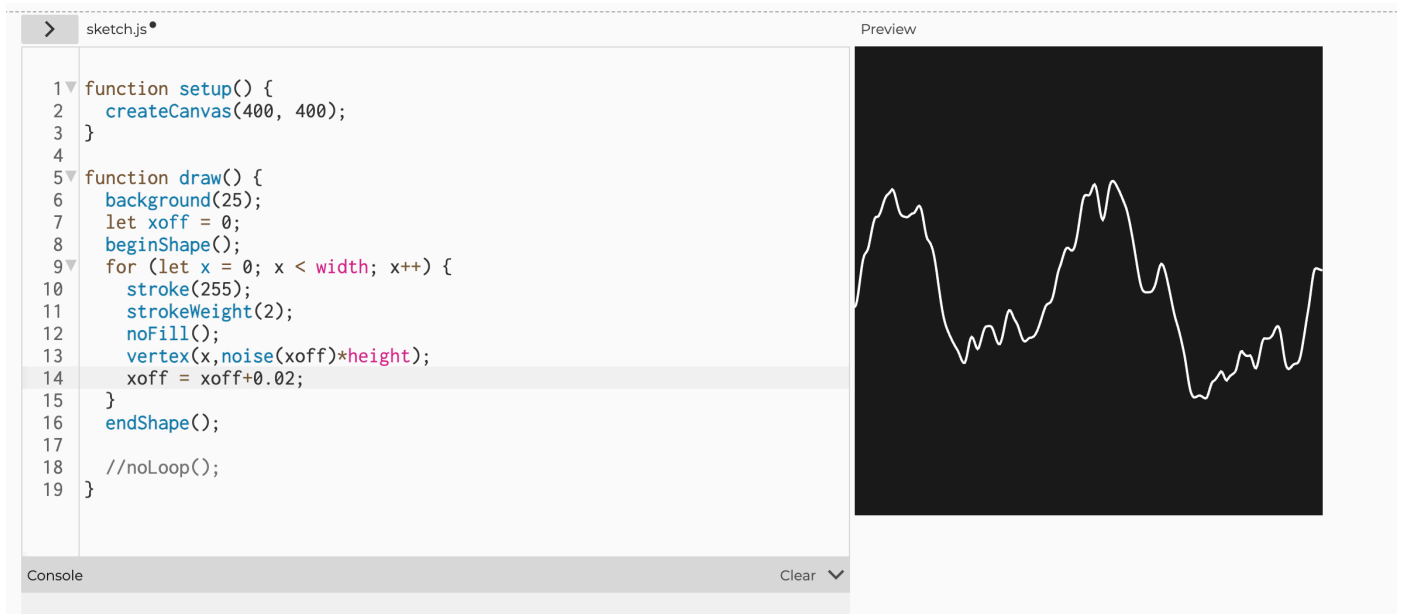
Preview

> sketch.js •

```
1 function setup() {
2   createCanvas(400, 400);
3 }
4
5 function draw() {
6   background(25);
7
8   beginShape();
9   for (let x = 0; x < width; x++) {
10    stroke(255);
11    strokeWeight(2);
12    noFill();
13    vertex(x, random(height));
14  }
15  endShape();
16
17  noLoop();
18 }
```

Console Clear ▼

Preview



So, now by starting off with a starting value, and updating that value each time by a small amount, I have used the noise function to give me a smooth correlated variation.

The ideas in 1-dimension can now be translated into two dimensions. Let's have a look at some examples here: <https://medium.com/@strawberry404/bridging-math-and-art-perlin-noise-p2-0718bd236dc2> [↗ \(https://medium.com/@strawberry404/bridging-math-and-art-perlin-noise-p2-0718bd236dc2\)](https://medium.com/@strawberry404/bridging-math-and-art-perlin-noise-p2-0718bd236dc2)

## 5. The use of Perlin noise in art and design

One of the most primary uses for Perlin noise is to directly use the spatial correlation analogy and generate artificial landscapes. For example, here is an artificial landscape generated using Perlin noise:



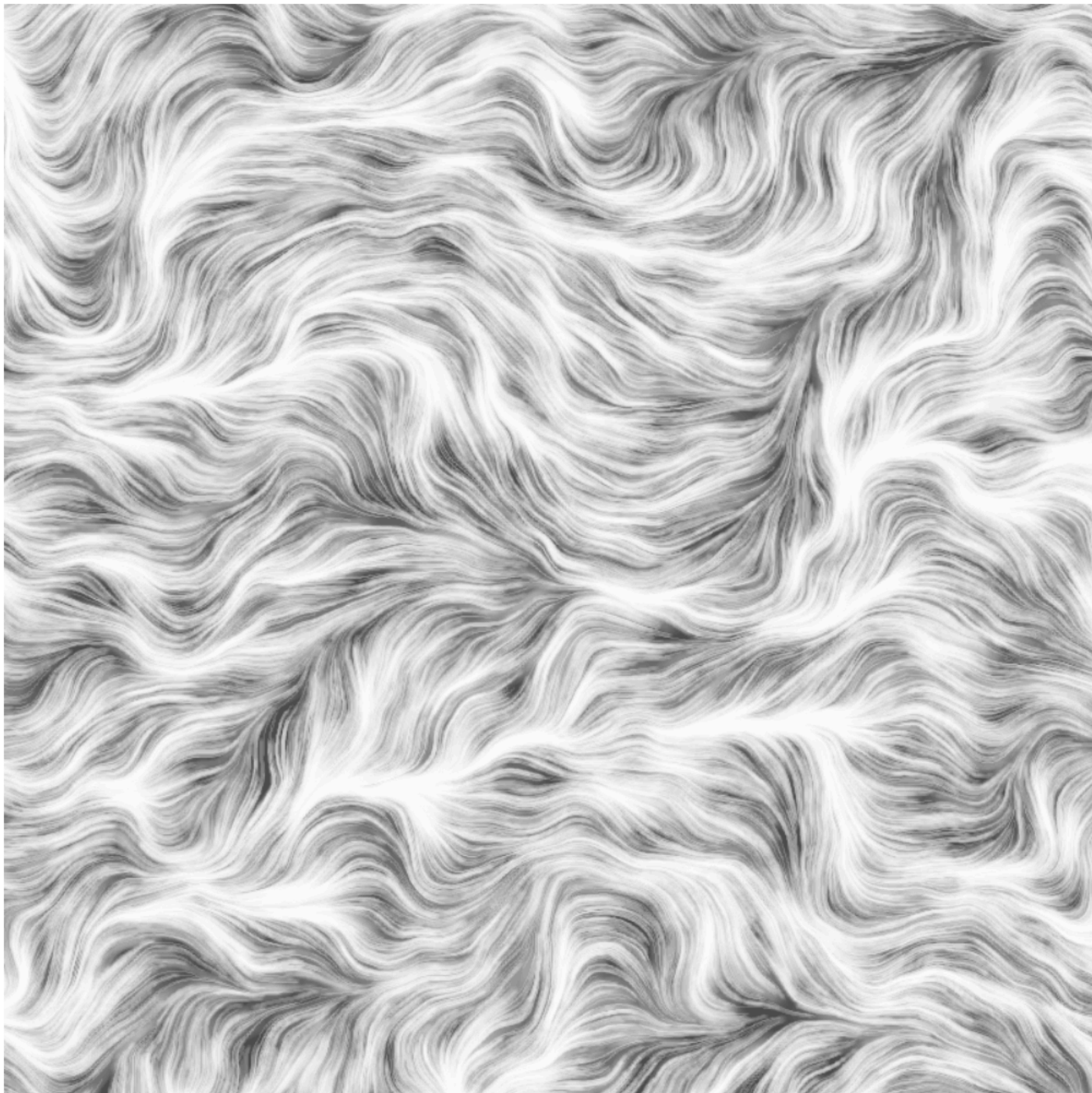
[https://en.wikipedia.org/wiki/Perlin\\_noise](https://en.wikipedia.org/wiki/Perlin_noise) [↗ \(https://en.wikipedia.org/wiki/Perlin\\_noise\)](https://en.wikipedia.org/wiki/Perlin_noise)

But, you can also do more abstract stuff, for example like combining math and art, like we saw in the link above.

And then, you can start to use abstraction to generate realistic looking effects:

<https://sighack.com/post/getting-creative-with-perlin-noise-fields> 

[\(https://sighack.com/post/getting-creative-with-perlin-noise-fields\)](https://sighack.com/post/getting-creative-with-perlin-noise-fields)





Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.  
[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)